Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

по лабораторной работе № 6

по курсу «Анализ алгоритмов»

Студент ИУ7-51Б (Группа) _____ (Подпись, дата) Платонова М. И. (И. О. Фамилия)

Преподаватель _____ (Подпись, дата) Волкова Л. Л. (И. О. Фамилия)

2024 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Аналитическая часть **4**

1.1 Задача Коммивояжёра 4

1.2 Метод полного перебора 4

1.3 Метод, основанный на муравьином алгоритме 4

2 Конструкторская часть **7**

2.1 Описание структуры ПО 7

2.2 Описание алгоритмов 8

2.3 Алгоритм взаимодействия с пользователем 8

3 Технологическая часть **13**

3.1 Средства реализации 13

3.2 Реализация алгоритмов 13

3.3 Тестирование результатов поиска пути 17

3.4 Результаты исследования 18

3.5 Пример работы программы 19

4 Исследовательская часть **20**

4.1 Технические характеристики 20

4.2 Результаты исследования 20

4.3 Результаты параметризации 21

ЗАКЛЮЧЕНИЕ **23**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **24**

ПРИЛОЖЕНИЕ А **25**

ВВЕДЕНИЕ

В задачах комбинаторной оптимизации часто нужно найти решение, которое удовлетворяет заданным условиям и при этом требует минимальных затрат времени и ресурсов. Одной из таких классических задач является задача коммивояжёра. Она заключается в поиске самого короткого маршрута, который проходит через все указанные города ровно один раз и возвращается в начальную точку.

Для её решения используют разные подходы, такие как алгоритм полного перебора и метод, основанный на муравьином алгоритме.

Алгоритм полного перебора находит точное решение, проверяя все возможные маршруты и выбирая среди них самый короткий. Однако этот метод ресурсоёмкий, при условии, если количество городов сильно возрастает.

Метод, основанный на муравьином алгоритме, основывается на наблюдении за поведением реальных муравьёв. В поисках пищи они оставляют на своём пути феромоны, которые привлекают других муравьёв, помогая им выбрать кратчайший маршрут. Если на пути встречается препятствие, муравьи ищут обходные пути. Те из них, кто случайно выбирает более короткий маршрут, двигаются быстрее и оставляют больше феромонов, таким образом привлекая остальных. Этот метод позволяет быстро получать решения, что делает его эффективным для практического применения.

Целью данной лабораторной работы является освоение особенностей и оценка эффективности двух методов решения задачи коммивояжёра: муравьиного алгоритма и алгоритма полного перебора.

Для достижения поставленной цели необходимо выполнить следующие задачи:

— исследовать особенности задачи коммивояжёра и её применения;

— рассмотреть и реализовать алгоритм полного перебора;

— рассмотреть и реализовать алгоритм муравьиного алгоритма;

— провести исследование производительности и эффективности реализованных алгоритмов при различных входных данных;

— сформулировать и представить результаты в отчёте о проведённой лабораторной работе.

1 Аналитическая часть

1.1 Задача Коммивояжёра

Задача коммивояжёра заключается в нахождении кратчайшего маршрута, который проходит через все заданные города, посещая каждый из них ровно один раз, и возвращается в исходную точку [1]. Эта задача может быть рассмотрена как поиск минимального пути в полном графе, где вершины представляют города, а рёбра — расстояния между ними.

Необходимо выбрать порядок посещения городов так, чтобы суммарное расстояние было минимальным. Задача состоит в нахождении данного порядка с учётом расстояний между городами.

1.2 Метод полного перебора

Метод полного перебора заключается в том, что исследуются все возможные маршруты, и выбирается наиболее оптимальный. Данный алгоритм гарантирует нахождение точного решения задачи.

- **Преимущество:** метод гарантирует нахождение наилучшего решения задачи коммивояжёра, так как перебираются все возможные варианты маршрутов.
- **Недостаток:** сложность метода составляет $O(n!)$, что приводит к экспоненциальному росту времени вычислений с увеличением числа городов.

Таким образом, несмотря на то, что метод полного перебора даёт точное решение, его высокая вычислительная сложность ограничивает его использование для задач с большим числом городов.

1.3 Метод, основанный на муравьином алгоритме

Основная идея муравьиного алгоритма заключается в моделировании поведения муравьиной колонии, которая решает задачу коммивояжёра в течение $t_{\max}$ суток. Каждый муравей действует автономно, но его поведение влияет на всю колонию через следы феромонов, оставляемые на пройденном пути [2]. Эти следы служат сигналом для других муравьёв, направляя их по

наиболее выгодным маршрутам. Такое коллективное поведение помогает колонии находить кратчайшие пути и адаптироваться к изменяющимся условиям окружающей среды.

Процесс решения задачи коммивояжёра с использованием муравьиного алгоритма включает следующие этапы:

— утром муравьи выходят из муравейника и распределяются по городам;

— в дневной фазе каждый муравей формирует полный маршрут;

— к закату муравьи возвращаются в муравейник, где обновляется наилучшие результаты и длина лучшего найденного маршрута L_{best};

— в ночной фазе обновляется матрица феромонов.

Каждый муравей обладает тремя свойствами:

— **зрение** — муравей k в городе i оценивает привлекательность η_{ij} дуги $i - j$ по формуле:

$$\eta_{ij} = \frac{1}{D_{ij}}, \tag{1.1}$$

где D_{ij} — метка дуги; D — матрица смежности графа.

— **память** — муравей запоминает посещённые города в день t в кортеж $J_k(t)$.

— **обоняние** — муравей чует концентрацию феромонов $\tau_{ij}(t)$ на дуге $i - j$ в день t.

  – ячейки матрицы феромонов τ_{ij} инициализируются константой;
  – значения τ_{ij} не обновляются в фазу дня.

Находясь в городе i, муравей k выбирает следующий город j с вероятностью, вычисляемой по формуле (1.2):

$$P_{ij,k}(t) = \begin{cases} 0, & \text{если } j \in J_k(t), \\ \dfrac{\eta_{ij}^{\alpha}\tau_{ij}(t)^{\beta}}{\sum\limits_{q \notin J_k(t)} \left(\eta_{iq}^{\alpha}\tau_{iq}^{\beta}\right)}, & \text{иначе.} \end{cases} \tag{1.2}$$

В формуле (1.2):

— α и β — коэффициенты жадности и стадности соответственно, причём $\alpha + \beta = 1$:

  – при $\alpha = 0$ и $\beta = 1$ поведение полностью стадное;

  – при $\alpha = 1$ и $\beta = 0$ поведение полностью жадное.

Обновление концентрации феромона τ_{ij} происходит ночью по формуле (1.3):

$$\tau_{ij}(t+1) = \tau_{ij}(t) \cdot (1-p) + \Delta\tau_{ij}(t), \tag{1.3}$$

где p — коэффициент испарения ($p \in (0,1)$).

Изменение концентрации феромона на завтра $\Delta\tau_{ij}(t)$ определяется формулой (1.4):

$$\Delta\tau_{ij}(t) = \sum_{k=1}^{N} \Delta\tau_{ij,k}(t), \tag{1.4}$$

где N — количество муравьёв.

Величина $\Delta\tau_{ij,k}(t)$ вычисляется по формуле (1.5):

$$\Delta\tau_{ij,k}(t) = \begin{cases} 0, & \text{если муравей } k \text{ в день } t \text{ не использовал дугу,} \\ \frac{Q}{L_k(t)}, & \text{иначе.} \end{cases} \tag{1.5}$$

В формуле (1.5) Q — дневная квота феромона одного муравья, $L_k(t)$ — длина маршрута муравья k в день t. Значение Q подбирается на тестовом графе следующим образом:

— на тестовом графе запускаются расчёты с различными значениями Q;

— оценивается качество найденных маршрутов при разных значениях Q;

— выбирается значение Q, обеспечивающее хорошие результаты при минимальном количестве итераций.

После выбора оптимального значения Q оно вручную задаётся в программе для проведения окончательных расчётов.

Если значение $\tau_{ij}(t)$ становится меньше минимального значения $\tau_{\min}$, оно откатывается к $\tau_{\min}$.

Модификация с элитными муравьями усиливает феромоны на дугах наилучших маршрутов. В этом случае формула (1.4) принимает вид:

$$\Delta\tau_{ij}(t) = \sum_{k=1}^{N} \Delta\tau_{ij,k}(t) + \Delta\tau_{\text{элит}}(t), \tag{1.6}$$

где

$$\Delta\tau_{\text{элит}}(t) = \begin{cases} 0, & \text{если дуга } i-j \text{ не принадлежит маршруту } L_{\text{best}}, \\ \frac{Q}{L_{\text{best}}}, & \text{иначе.} \end{cases} \tag{1.7}$$

Вывод

В данном разделе рассмотрены алгоритмы решения задачи коммивояжёра: метод полного перебора и метод, основанный на муравьином алгоритме. Метод полного перебора обеспечивает нахождение оптимального пути, но его применение ограничено высокой вычислительной сложностью. Метод, основанный на муравьином алгоритме, хотя и позволяет быстро находить решения, обеспечивает более быстрое решение, что делает его предпочтительным для задач с большим числом городов.

2 Конструкторская часть

В данном разделе представлены описания алгоритмов решения задачи коммивояжёра: метод полного перебора и метод, основанный на муравьином алгоритме. Описана структура программного обеспечения, а также алгоритмы, реализующие данные методы, и их взаимодействие с пользователем.

2.1 Описание структуры ПО

На вход программного продукта подаётся взвешенный граф в виде матрицы смежности. Метод, основанный на муравьином алгоритме также принимает начальные значения феромонов, максимальное количество итераций, коэффициенты α и β, а также коэффициент испарений. Программа

позволяет пользователю выбрать один из двух алгоритмов: метод полного перебора или метод, основанный на муравьином алгоритме.

2.2 Описание алгоритмов

В программном продукте реализованы два алгоритма для решения задачи коммивояжёра.

— **Метод полного перебора**, который анализирует все возможные маршруты и выбирает оптимальный. Данный метод гарантирует нахождение наилучшего решения, но имеет высокую вычислительную сложность.

— **Метод, основанный на муравьином алгоритме**, который использует вероятностный подход для нахождения решения. Данный метод позволяет быстро находить решения, но не всегда гарантирует оптимальный путь.

Метод, основанный на муравьином алгоритме реализуется с учётом коэффициентов жадности α и стадности β, что влияет на поведение муравьёв при выборе путей. Алгоритм работает итеративно, обновляя значения феромонов и выбирая пути с учётом вероятностного распределения.

2.3 Алгоритм взаимодействия с пользователем

Программа взаимодействует с пользователем через консоль. Процесс работы с программой следующий.

Пользователь выбирает режим ввода данных:

— из файла;

— из консоли.

В случае выбора ввода из файла, программа запрашивает имя файла и загружает данные о графе (количество городов, количество дорог, данные о дорогах).

В случае выбора консольного ввода, программа запрашивает количество городов и количество дорог и информацию о каждой дороге (стартовый и конечный города, вес дороги).

Далее пользователю предлагается выбрать алгоритм для решения задачи:

— метод, основанный на муравьином алгоритме (Ant Colony Optimization);

— метод полного перебора (Brute Force).

Если выбран метод, основанный на муравьином алгоритме, программа запрашивает дополнительные параметры:

— количество итераций;

— количество муравьёв в каждом городе;

— количество элитных муравьёв;

— коэффициент испарения феромонов;

— значения феромонов для общего пути и для элитных муравьёв;

— коэффициенты для феромонов и веса пути.

Если выбран метод полного перебора, программа сразу выполняет поиск наилучшего пути, анализируя все возможные маршруты.

В обоих случаях программа выводит наилучший найденный путь, его длину, порядок посещённых городов и время выполнения алгоритма.

На рисунках 2.1 и 2.2 приведены схемы реализации алгоритмов полного перебора и муравьиного алгоритма.

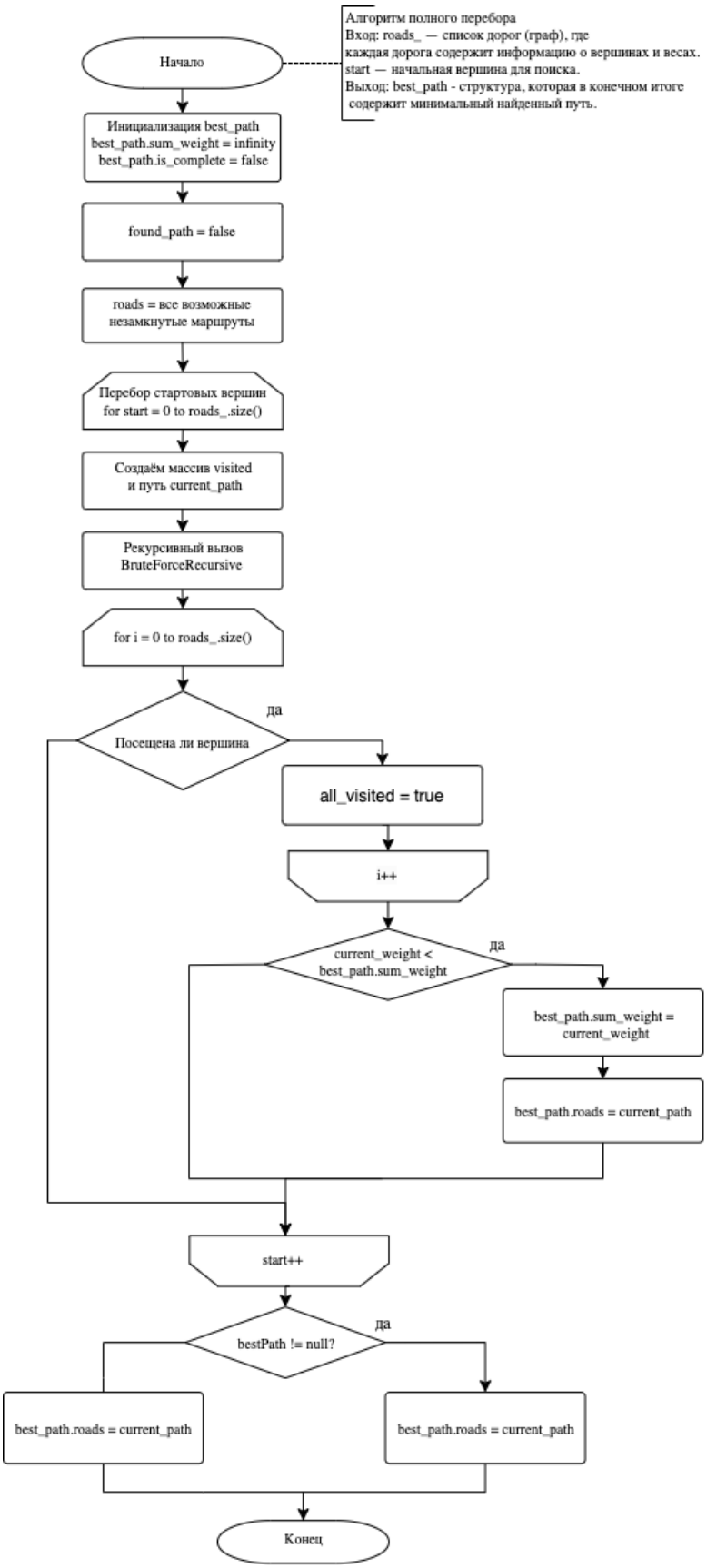


Рисунок 2.1 – Схема алгоритма полного перебора

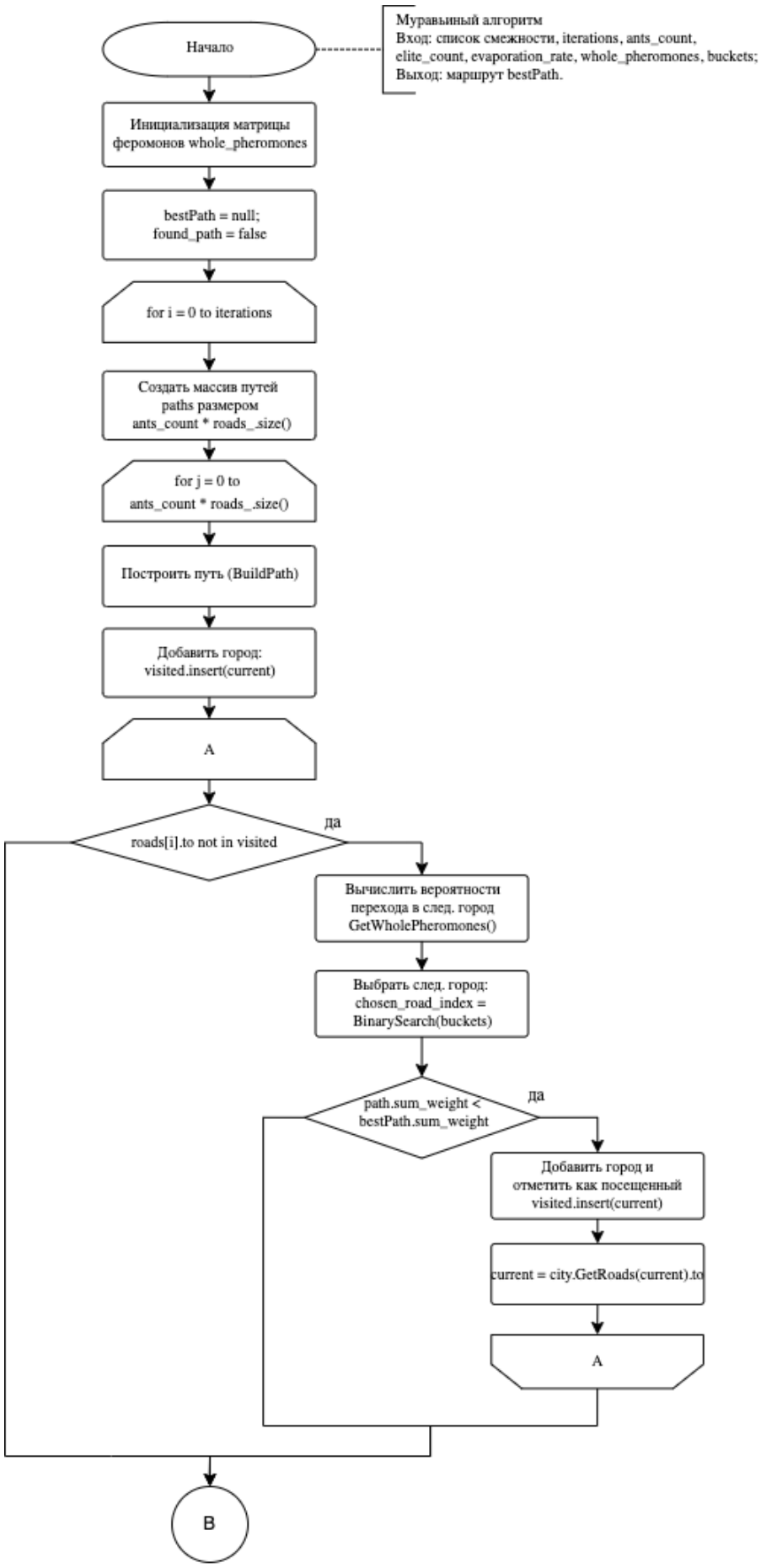


Рисунок 2.2 – Схема муравьиного алгоритма (начало)

Рисунок 2.3 – Схема муравьиного алгоритма (продолжение)

3 Технологическая часть

В данном разделе приведены данные о выбранном языке программирования и реализации алгоритмов поиска.

3.1 Средства реализации

Для реализации алгоритмов поиска, проведения замеров времени и обработки данных в данной работе используется язык программирования `C++` [3]. Язык предоставляет мощные инструменты для работы с массивами, а также позволяет эффективно управлять временем выполнения с помощью различных библиотек.

3.2 Реализация алгоритмов

В листинге 3.1 представлена функция `ChooseRoad` для выбора дороги в муравьином алгоритме.

Листинг 3.1 – Метод выбора дороги для муравья

```
size_t Ant::ChooseRoad(const std::vector<Road>& roads, double
   ph_pow, double weight_pow,
                        std::unordered_set<size_t>& visited) {
  std::vector<size_t> roads_indices;
  for (size_t i = 0; i < roads.size(); ++i) {
    if (visited.find(roads[i].to) == visited.end()) {
      roads_indices.push_back(i);
    }
  }
  if (roads_indices.empty()) {
    return roads.size();
  }
  double whole_pheromones = GetWholePheromones(roads_indices,
     roads, ph_pow, weight_pow);
  std::vector<double> buckets(roads_indices.size() + 1);
  buckets[0] = 0;
  for (size_t i = 1; i < buckets.size(); ++i) {
    buckets[i] = buckets[i - 1] +
       GetCoefficient(roads[roads_indices[i - 1]], ph_pow,
       weight_pow);
```

```
  }
  for (int i = 0; i < buckets.size(); ++i) {
    buckets[i] /= whole_pheromones; }
  std::random_device rd;
  std::mt19937 gen(rd());
  std::uniform_real_distribution<> dis(0, buckets.back());
  double value = dis(gen);
  return roads_indices[BinarySearch(buckets, value) - 1];
}
```

В листинге 3.2 представлена функция `BuildPath` для построения пути для муравья.

Листинг 3.2 – Метод построения пути для муравья

```
Path Ant::BuildPath(City& city, size_t start, double ph_pow,
   double weight_pow) {
  Path path;
  path.is_complete = false;
  size_t current = start;
  std::unordered_set<size_t> visited;
  visited.insert(current);
  path.sum_weight = 0;
  while (visited.size() < city.GetSize()) {
    size_t chosen_road_index =
       ChooseRoad(city.GetRoads(current), ph_pow, weight_pow,
       visited);
    if (chosen_road_index == city.GetRoads(current).size()) {
      return path; }
    path.roads.push_back(
        &city.GetRoads(current)[chosen_road_index]);
    path.sum_weight +=
       city.GetRoads(current)[chosen_road_index].weight;
    current = city.GetRoads(current)[chosen_road_index].to;
    visited.insert(current); }
  path.is_complete = true;
  path.start = start;
  return path; }
```

В листинге 3.3 представлена функция `Crowl` для муравьиного алгоритма.

Листинг 3.3 – Метод муравьиного алгоритма для поиска пути

```
Path City::Crowl(size_t iterations, size_t ants_count, size_t
   elite_count, double evaporation_rate,
                double common_pheromones, double
                   elite_pheromones, double ph_pow, double
                   weight_pow) {
  Path best_path;
  bool found_path = false;
  for (size_t i = 0; i < iterations; ++i) {
    std::vector<Path> paths(ants_count * roads_.size());
    for (size_t j = 0; j < ants_count * roads_.size(); ++j) {
      size_t start = j % roads_.size();
      paths[j] = Ant().BuildPath(*this, start, ph_pow,
         weight_pow);
    }
    for (auto& path : paths) {
      for (auto& road : path.roads) {
        road->pheromones *= (1 - evaporation_rate);
      }
    }
    std::sort(paths.begin(), paths.end(), [](const Path& a,
       const Path& b) {
      if (a.is_complete && !b.is_complete) {
        return true;
      }
      if (!a.is_complete && b.is_complete) {
        return false;
      }
      return a.sum_weight < b.sum_weight;
    });
    for (size_t j = 0; j < elite_count; ++j) {
      if (!paths[j].is_complete) {
        continue;
      }
      for (auto& road : paths[j].roads) {
        road->elite_visited = true;
      }
```

```
    }
    for (size_t j = elite_count; j < paths.size(); ++j) {
      if (!paths[j].is_complete) {
        continue;
      }
      for (auto& road : paths[j].roads) {
        road->visited = true;
      }
    }
    for (auto& path : paths) {
      for (auto& road : path.roads) {
        if (road->elite_visited) {
          road->pheromones += elite_pheromones;
        } else if (road->visited) {
          road->pheromones += common_pheromones;
        }
        road->elite_visited = false;
        road->visited = false;
      }
    }
    if (paths[0].is_complete && (!found_path ||
       paths[0].sum_weight < best_path.sum_weight)) {
      best_path = paths[0];
      found_path = true;
    }
  }
  return best_path;
}
```

В листинге 3.4 представлена функция `BruteForce` для решения задачи коммивояжёра методом полного перебора.

Листинг 3.4 – Метод полного перебора для решения задачи коммивояжёра

```
Path City::BruteForce() {
  Path best_path;
  best_path.sum_weight =
     std::numeric_limits<double>::infinity();
```

```
best_path.is_complete = false;
bool found_path = false;
for (size_t start = 0; start < roads_.size(); ++start) {
  std::vector<size_t> cities(roads_.size());
  for (size_t i = 0; i < cities.size(); ++i) {
    cities[i] = i;}
  cities.erase(std::find(cities.begin(), cities.end(),
     start));
  do {
    Path path;
    path.is_complete = true;
    path.sum_weight = 0;
    size_t current = start;
    for (size_t i = 0; i < cities.size(); ++i) {
      size_t next = cities[i];
      auto it = std::find_if(roads_[current].begin(),
         roads_[current].end(),
                              [next](const Road& r) { return
                                 r.to == next; });
      if (it != roads_[current].end()) {
        path.roads.push_back(&(*it));
        path.sum_weight += it->weight;
        current = next;
      } else {
        path.is_complete = false;
        break;
      }
    }
    if (path.is_complete && path.sum_weight <
       best_path.sum_weight) {
      best_path = path;
    }
  } while (std::next_permutation(cities.begin(),
     cities.end())); }
return best_path; }
```

3.3 Тестирование результатов поиска пути

В таблице 3.1 приведены результаты функционального тестирования для алгоритмов поиска пути, использующих полный перебор и метод, основанный

на муравьином алгоритме. Оба метода тестировались на одинаковых входных данных для проверки корректности их работы. Основное различие в подходе: полный перебор гарантирует оптимальное решение, а муравьиный алгоритм предоставляет решение за меньшее время.

Таблица 3.1 – Функциональные тесты алгоритмов поиска пути

№ теста	Входные данные	Ожидаемый результат	Результат теста
1	Матрица смежности: $\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & 6 & 7 & 8 & 9 \\ 2 & 6 & 0 & 10 & 11 & 12 \\ 3 & 7 & 10 & 0 & 13 & 14 \\ 4 & 8 & 11 & 13 & 0 & 15 \\ 5 & 9 & 12 & 14 & 15 & 0 \end{pmatrix}$	3.5, (0, 3, 1, 6, 5, 2, 4)	Успех (Решение найдено с использованием метода, основанного на муравьином алгоритме)
2	Матрица смежности: $\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 1 & 0 & 6 & 7 & 8 & 9 \\ 2 & 6 & 0 & 10 & 11 & 12 \\ 3 & 7 & 10 & 0 & 13 & 14 \\ 4 & 8 & 11 & 13 & 0 & 15 \\ 5 & 9 & 12 & 14 & 15 & 0 \end{pmatrix}$	3.5, (0, 3, 1, 6, 5, 2, 4)	Успех (Решение найдено с использованием метода полного перебора)

3.4 Результаты исследования

Тестирование подтвердило корректность асимптотической сложности алгоритмов:

— метод на основе муравьиного алгоритма имеет сложность $O(I \cdot A \cdot (V+E))$, где I — количество итераций, A — количество муравьёв, V — число вершин, E — число рёбер.

— алгоритм полного перебора имеет сложность $O(V!)$, так как количество перестановок вершин растёт факториально.

Измерение времени выполнения относится не к самим алгоритмам, а к их программной реализации, которая зависит от платформы, кода и условий тестирования. На рисунке 4.1 представлено сравнение времени работы двух методов в зависимости от размера графа. Это сравнение показывает практическую эффективность программной реализации для каждого метода.

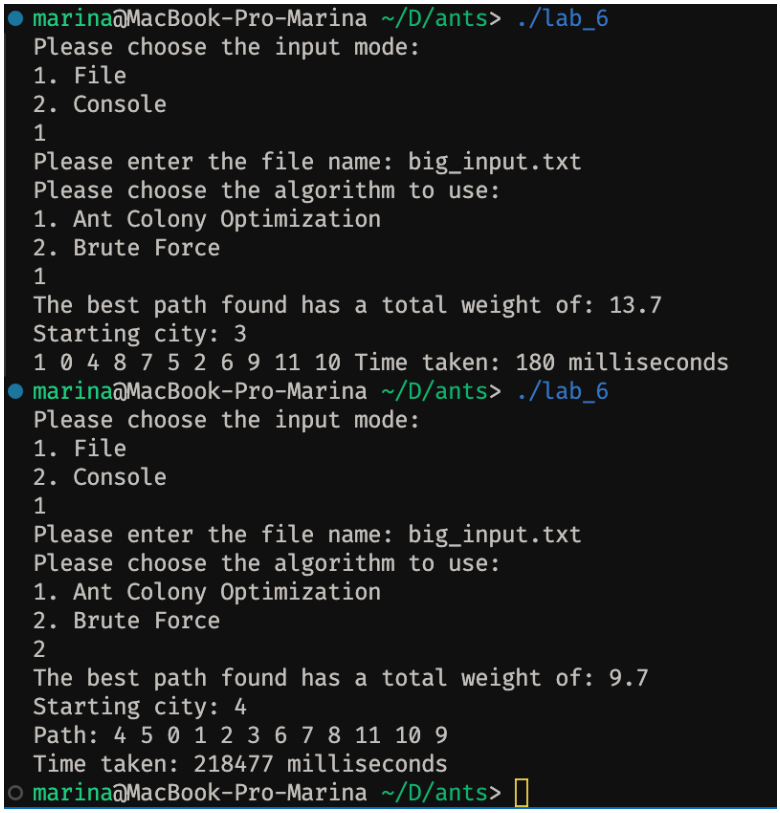
Основные различия тестов для двух методов:

— метод полного перебора гарантирует нахождение оптимального решения, но требует больше времени для больших графов;

— метод на основе муравьиного алгоритма быстрее, но может предоставлять решение, что подходит для задач, где допустима погрешность.

3.5 Пример работы программы

На рисунке 3.1 представлена демонстрация работы программы.



```
marina@MacBook-Pro-Marina ~/D/ants> ./lab_6
Please choose the input mode:
1. File
2. Console
1
Please enter the file name: big_input.txt
Please choose the algorithm to use:
1. Ant Colony Optimization
2. Brute Force
1
The best path found has a total weight of: 13.7
Starting city: 3
1 0 4 8 7 5 2 6 9 11 10 Time taken: 180 milliseconds
marina@MacBook-Pro-Marina ~/D/ants> ./lab_6
Please choose the input mode:
1. File
2. Console
1
Please enter the file name: big_input.txt
Please choose the algorithm to use:
1. Ant Colony Optimization
2. Brute Force
2
The best path found has a total weight of: 9.7
Starting city: 4
Path: 4 5 0 1 2 3 6 7 8 11 10 9
Time taken: 218477 milliseconds
marina@MacBook-Pro-Marina ~/D/ants>
```

Рисунок 3.1 – Демонстрация работы программы

Вывод

В данном разделе представлены результаты тестирования алгоритмов поиска пути. Все тесты пройдены успешно, фактические результаты совпадают

с ожидаемыми.

4 Исследовательская часть

В данном разделе представлены результаты исследования эффективности работы алгоритмов по времени выполнения, основанной на критерии временных затрат.

4.1 Технические характеристики

Технические характеристики компьютера, на котором проводились замеры времени:

- операционная система: macOS Monterey;
- оперативная память: 8 ГБ;
- процессор: Quad-Core Intel Core i5;

4.2 Результаты исследования

В таблице 4.1 представлено время выполнения алгоритмов Ant Colony Optimization и Brute Force для графов с различным количеством вершин, измеренное в миллисекундах.

Для каждого графа и каждой тройки параметров муравьиного алгоритма (I — число итераций, A — количество муравьёв, ρ — коэффициент испарения феромонов) было выполнено 50 измерений времени работы для каждого из алгоритмов. Результаты измерений усреднялись. Время замерялось с помощью встроенной функции времени в языке C++.

Алгоритм полного перебора был значительно медленнее, чем метод, основанный на муравьином алгоритме, особенно при увеличении количества вершин в графе.

На рисунке 4.1 показано, как изменяется время выполнения алгоритмов в зависимости от количества вершин для метода, основанном на муравьином алгоритме и метода полного перебора.

Таблица 4.1 – Время выполнения алгоритмов в зависимости от входных данных (в миллисекундах)

Количество вершин	Алгоритм полного перебора	Муравьиный алгоритм
3	0	0
4	0	1
5	1	7
6	2	7
7	2	17
9	53	20
10	200	31
11	913	32

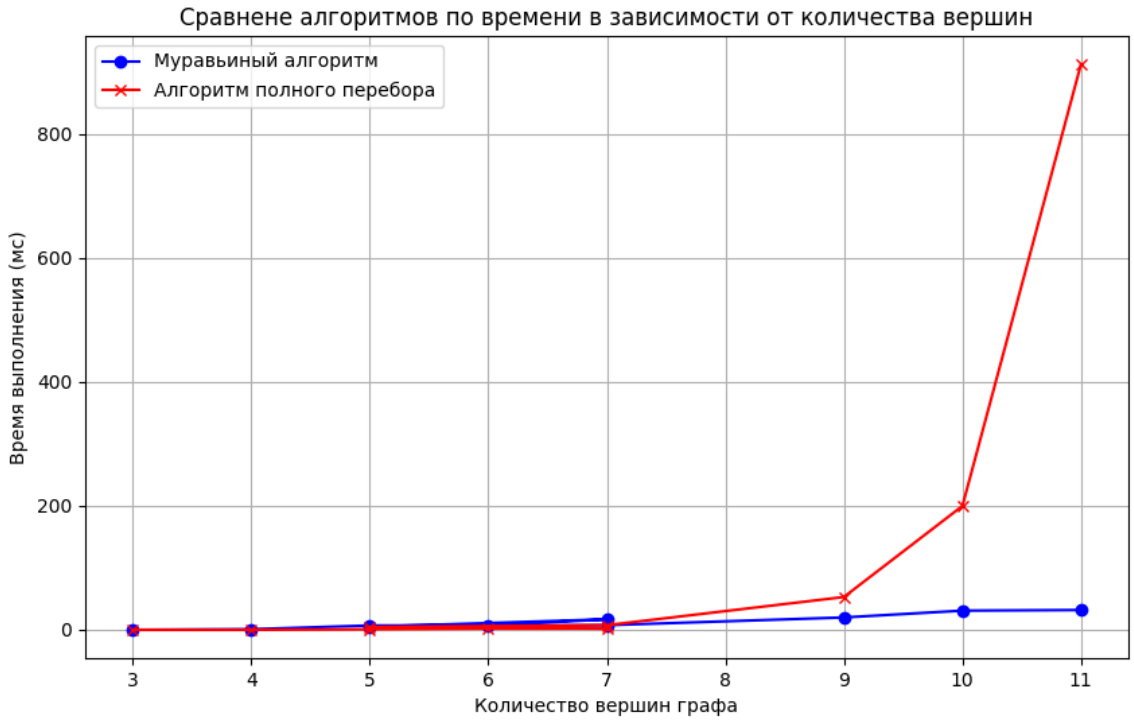


Рисунок 4.1 – Зависимость времени выполнения алгоритмов от количества вершин

4.3 Результаты параметризации

В ходе параметризации исследовались различные комбинации параметров, таких как количество итераций (I), количество муравьёв (A) и коэффициент испарения феромонов (ρ), с целью оптимизации работы алгоритма. Результаты приведены в таблице, где указаны максимальные, медианные и средние значения отклонений стоимости маршрута от эталонного.

Результаты параметризации показывают, что метод, основанный на

Таблица 4.2 – Время выполнения алгоритмов в зависимости от параметров (в миллисекундах)

α	ρ	I	max_t1	avg_t1	med_t1	max_t2	avg_t2	med_t2
0.5	0.3	50	3.10	0.95	0.05	6.10	5.20	6.05
1.0	0.3	50	0.10	0.05	0.00	9.10	7.00	6.10
1.5	0.3	50	0.00	0.00	0.00	9.10	7.60	7.55
2.0	0.1	100	1.05	0.15	0.05	6.05	4.25	4.55
2.5	0.1	100	0.00	0.00	0.00	9.05	6.65	6.05
1.0	0.3	100	0.00	0.00	0.00	9.05	7.55	9.05

муравьином алгоритме значительно сокращает время работы для небольших графов.

Вывод

Результаты исследования показывают значительное различие в времени выполнения между двумя алгоритмами. Метод, основанный на муравьином алгоритме показал значительное сокращение времени выполнения для небольших графов, но с увеличением количества вершин время выполнения увеличивается. Алгоритм полного перебора демонстрирует резкое увеличение времени при росте числа вершин, что делает его менее эффективным для больших графов.

Например, для графа с 5 вершинами время работы муравьиного алгоритма составило 7 миллисекунд, в то время как алгоритм полного перебора потребовал 200 миллисекунд. Для графов с 11 вершинами время работы муравьиного алгоритма всего 32 миллисекунды, тогда как алгоритм полного перебора занял 913 миллисекунд.

Таким образом, метод, основанный на муравьином алгоритме наиболее эффективен для задач с большим количеством вершин, так как его время выполнения значительно ниже по сравнению с алгоритмом полного перебора, который становится менее эффективным при увеличении размера графа.

ЗАКЛЮЧЕНИЕ

Цель лабораторной работы была достигнута. В ходе выполнения работы решены следующие задачи:

- исследованы особенности задачи коммивояжёра и её применения, включая трудности нахождения оптимального пути для большого числа городов;
- рассмотрен и реализован алгоритм полного перебора, который обеспечивает точное решение, но имеет высокие вычислительные затраты при увеличении количества городов;
- рассмотрен и реализован метод, основанный на муравьином алгоритме, который является эвристическим методом, способным находить решения с меньшими вычислительными затратами;
- проведено исследование производительности и эффективности реализованных алгоритмов при различных входных данных;
- сформулированы и представлены результаты в отчёте о проведённой лабораторной работе.

Исследования показали, что метод, основанный на муравьином алгоритме значительно быстрее, особенно при решении задач с большим числом вершин. В отличие от алгоритма полного перебора, который даёт точные результаты, но его время выполнения резко увеличивается с ростом количества вершин, что делает его трудным для использования при решении больших задач. Метод, основанный на муравьином алгоритме, в свою очередь, сохраняет хорошую эффективность и существенно ускоряет поиск решения, оставаясь эффективным даже для крупных графов.

Таким образом, для задач, где важна скорость нахождения решения, особенно при большом числе вершин, метод, основанный на муравьином алгоритме является предпочтительным. Алгоритм полного перебора лучше подходит для небольших и средних графов, где важна точность результата.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Штовба С.* Муравьиные алгоритмы // Exponenta Pro. Математика в приложениях. — 2003. — № 4. — С. 70—75.
2. *Ульянов М.* РЕСУРСНО-ЭФФЕКТИВНЫЕ КОМПЬЮТЕРНЫЕ АЛГОРИТМЫ. РАЗРАБОТКА И АНАЛИЗ. — Москва : НАУКА ФИЗМАТЛИТ, 2007.
3. *Страуструп Б.* Язык программирования C++. — Москва : Вильямс, 2004. — С. 1024. — ISBN 5-8459-0200-1.

ПРИЛОЖЕНИЕ А

Результаты параметризации